

RAJARATA UNIVERSITY OF SRI LANKA



Department of Information & Communication Technology
ICT 1209 – Web Technologies

Mini Project Final Report

Student Notes Management System

Theme: Student Resource Hub — Academic Resource Sharing & Collaborative Student Tools

Name	Registration No.	Index No.
D.I.A.A. Wijesekara	ITT/2024/121	2806
K.S. Liyanage	ITT/2024/126	2811

Batch: 2024 | Bachelor of ICT (First Year)
Submitted for: Phase 3 – PHP & MySQL Integration (Final Submission)
Submission Date: Sunday, 7th September 2026
GitHub Repository: https://github.com/Inuka-Asith30/Student_Resource_Hub

Table of Contents

Table of Contents.....	1
1. Project Overview	2
1.1 Purpose	2
1.2 Target Audience.....	2
1.3 Key Features	2
2. System Design.....	2
2.1 Technology Stack	2
2.2 Database Schema	2
users	3
messages.....	3
notes (theme-specific table)	3
2.3 Entity Relationship Overview.....	3
2.4 Folder Structure	4
3. Implementation	4
3.1 User Authentication	4
3.2 Dashboard & Personalisation.....	5
3.3 Notes Repository & Real-time Search.....	6
3.4 Note Upload with Dual-side Validation	7
3.5 Navigation & Branding Clean-up.....	7
3.6 Security Measures	7
4. Challenges and Solutions.....	7
5. Individual Contribution	8
6. Setup Instructions.....	9
7. References	9

1. Project Overview

1.1 Purpose

The Student Notes Management System (developed under the theme “Student Resource Hub”) is a web-based platform built to close the resource gap that first-year and continuing university students face when trying to find organised study material. Rather than notes being scattered across private chats and messaging groups, the application gives students a single, searchable place to upload, browse, and download academic notes shared by their peers, presented under the branded “Campus Portal” interface.

1.2 Target Audience

- Primary audience: undergraduate and postgraduate university students who need quick access to lecture notes, summaries, and past academic material.
- Secondary audience: lecturers and teaching assistants who may wish to reference or contribute to the shared resource pool.

1.3 Key Features

- Secure user registration and login with hashed passwords and session-based authentication, with automatic sign-in immediately after registration.
- Personalised dashboard that greets each student by name and summarises platform-wide and user-specific statistics (total notes, subjects available, notes uploaded/downloaded).
- Searchable past-papers repository organised into subject cards, with a live JavaScript filter that narrows results as the user types — no page reload required.
- Note upload module with subject, year and PDF-only file validation on both the client (JavaScript) and server (PHP) side.
- Custom Campus Portal branding, including a university-style crest logo used consistently across the login, registration and sidebar navigation.
- Streamlined, role-appropriate navigation — the authenticated sidebar shows only Dashboard, Notes and Upload Notes, keeping logged-in students focused on core tasks.

2. System Design

2.1 Technology Stack

Layer	Technology
Frontend Structure	HTML5, CSS3, Bootstrap 5
Client-side Logic	JavaScript (Vanilla)
Backend	PHP 8
Database	MySQL (via XAMPP)
Version Control	Git & GitHub

2.2 Database Schema

The application database, `student_resource_hub`, contains three related tables: `users`, `messages` and `notes`. The `notes` table stores a foreign key reference to the uploading user, and cascades on delete so orphaned records are not left behind if a user account is removed. The `messages` table remains part of the schema for extensibility, though the current interface routes all interaction through the notes workflow.

users

Field	Type	Notes
id	INT, AUTO_INCREMENT, PRIMARY KEY	Unique user identifier
username	VARCHAR(50), NOT NULL	Display name shown on dashboard and notes pages
email	VARCHAR(100), NOT NULL, UNIQUE	Login identifier
password	VARCHAR(255), NOT NULL	Hashed with password_hash() / BCrypt
created_at	TIMESTAMP, DEFAULT CURRENT_TIMESTAMP	Registration time

messages

Field	Type	Notes
id	INT, AUTO_INCREMENT, PRIMARY KEY	Unique message identifier
name	VARCHAR(100), NOT NULL	Sender name
email	VARCHAR(100), NOT NULL	Sender email
message	TEXT, NOT NULL	Message body
created_at	TIMESTAMP, DEFAULT CURRENT_TIMESTAMP	Submission time

notes (theme-specific table)

Field	Type	Notes
id	INT, AUTO_INCREMENT, PRIMARY KEY	Unique note identifier
user_id	INT, NOT NULL, FOREIGN KEY → users(id)	Uploader; ON DELETE CASCADE
subject_name	VARCHAR(100), NOT NULL	Subject label shown on the card
note_year	INT, NOT NULL	Academic year of the note
file_path	VARCHAR(255), NOT NULL	Server path to the stored PDF
created_at	TIMESTAMP, DEFAULT CURRENT_TIMESTAMP	Upload time

2.3 Entity Relationship Overview

Each note belongs to exactly one user (one-to-many: one user can upload many notes), and the dashboard, notes and upload pages all join against the users table to resolve the logged-in student's display name from their session id. The messages table remains independent and is not linked to a specific user account.

```

users (1) ———< (many) notes
  id          user_id (FK)

messages (standalone log table)
```

2.4 Folder Structure

The project follows the folder structure specified in the mini project guide, with a dedicated `images/` directory added for the project's branding asset:

```
Student_Resource_Hub_2/
├── css/           → compiled stylesheet output
├── js/           → script.js (validation, search, animations)
├── images/
│   └── logo.png   → Campus Portal crest logo
├── uploads/      → student-submitted PDF notes
├── includes/
│   ├── db.php     → PDO database connection
│   └── functions.php → session helpers (isLoggedIn, requireLogin)
├── auth/
│   ├── register.php
│   ├── login.php
│   └── logout.php
├── pastpapers.php → notes repository (formerly notes.php)
├── upload.php
├── index.php
├── dashboard.php
├── README.md
└── database.sql
```

3. Implementation

This section walks through each major feature of the system, how it was implemented, and a screenshot of the working page.

3.1 User Authentication

Registration (`auth/register.php`) validates that all fields are present and that the password is at least 6 characters, checks the email is not already registered, then stores the account with a BCrypt hash produced by PHP's `password_hash()`. On success the user is now signed in immediately: the new account's session is started with `session_regenerate_id(true)` and `$_SESSION['user_id']`, and the browser is redirected straight to the dashboard instead of back to the login page. Login (`auth/login.php`) looks up the account by email, verifies the password using `password_verify()`, and likewise calls `session_regenerate_id(true)` to prevent session-fixation attacks before storing the user's id in the session. Both pages also set an `isLoggedIn` flag in the browser's `localStorage` as they redirect, keeping the client-side navigation script in sync with the server-side session. Logout (`auth/logout.php`) clears and destroys the session cleanly.

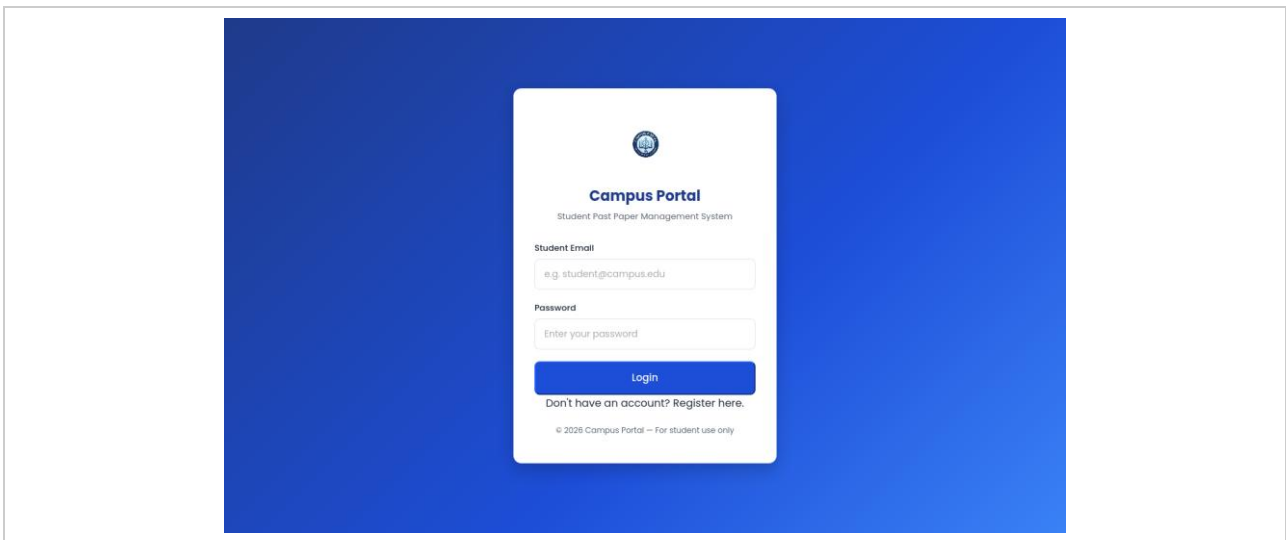


Figure 1 — Login page with the Campus Portal crest logo

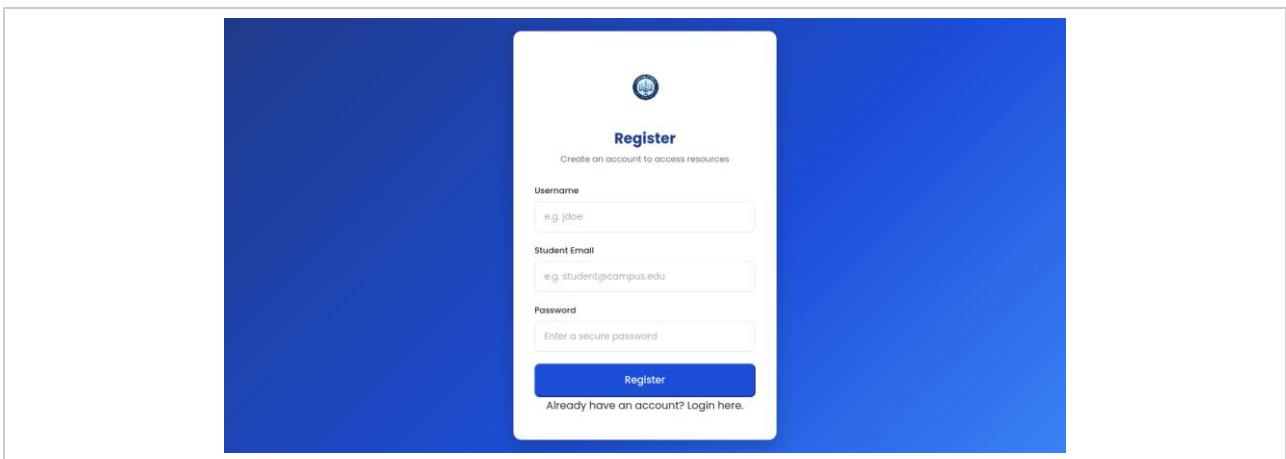


Figure 2 — Registration page with client-side validation

3.2 Dashboard & Personalisation

dashboard.php requires an active session (requireLogin()) and now queries the users table for the logged-in student's username, deriving a two-letter avatar initial from it. The welcome banner and the top-right user chip both address the student by name rather than showing a generic placeholder. The stat panel queries the notes table with COUNT(*) for a live Total Notes figure, alongside Subjects Available, Notes Downloaded and Notes Uploaded metrics matching the approved wireframe, plus two quick-action shortcuts to the Notes and Upload pages.

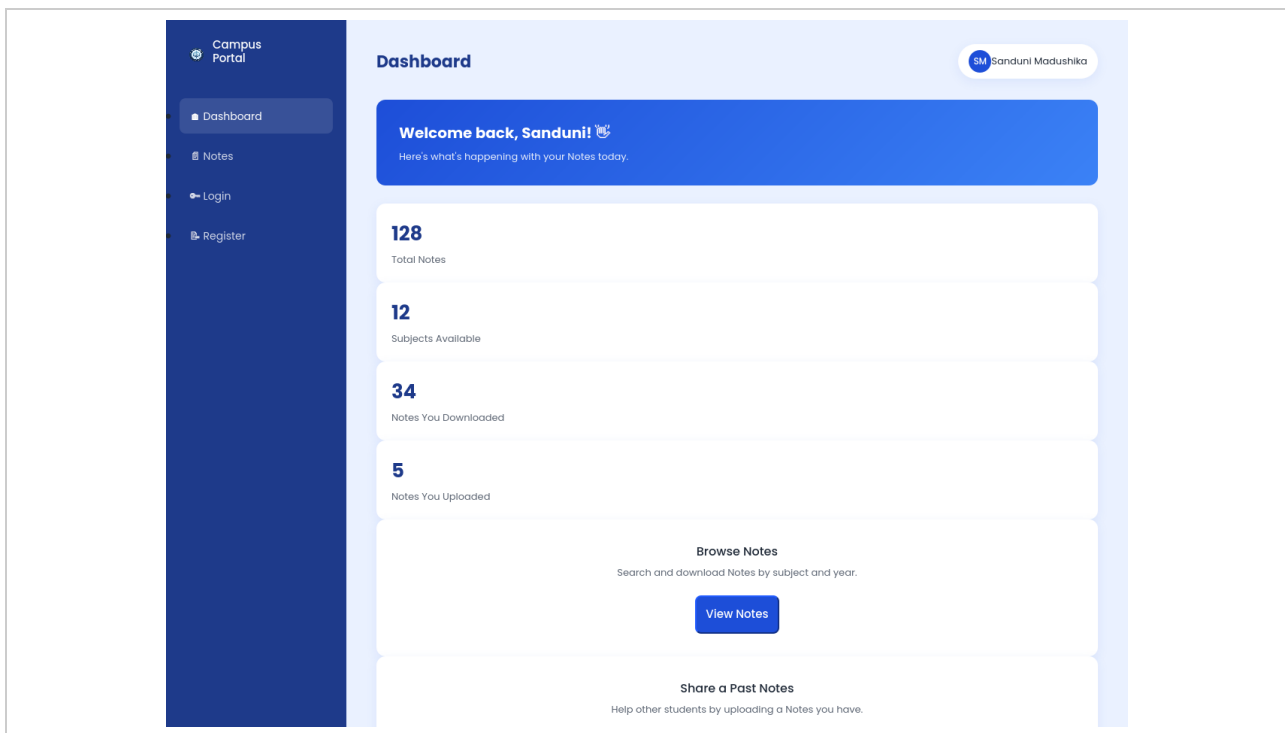


Figure 3 — Personalised dashboard with live note statistics

3.3 Notes Repository & Real-time Search

The notes repository page was renamed from notes.php to pastpapers.php to better match the project's terminology, and its route is updated throughout the sidebar and the dashboard's quick-action buttons. It fetches every row from the notes table (ordered by upload date) and renders a Bootstrap card grid, tagging each card with a data-subject attribute. The accompanying script.js listens for input on the search box and hides any card whose subject or heading does not contain the typed text — filtering the grid instantly without a page reload, exactly as specified in the Phase 2 dynamic content requirement.

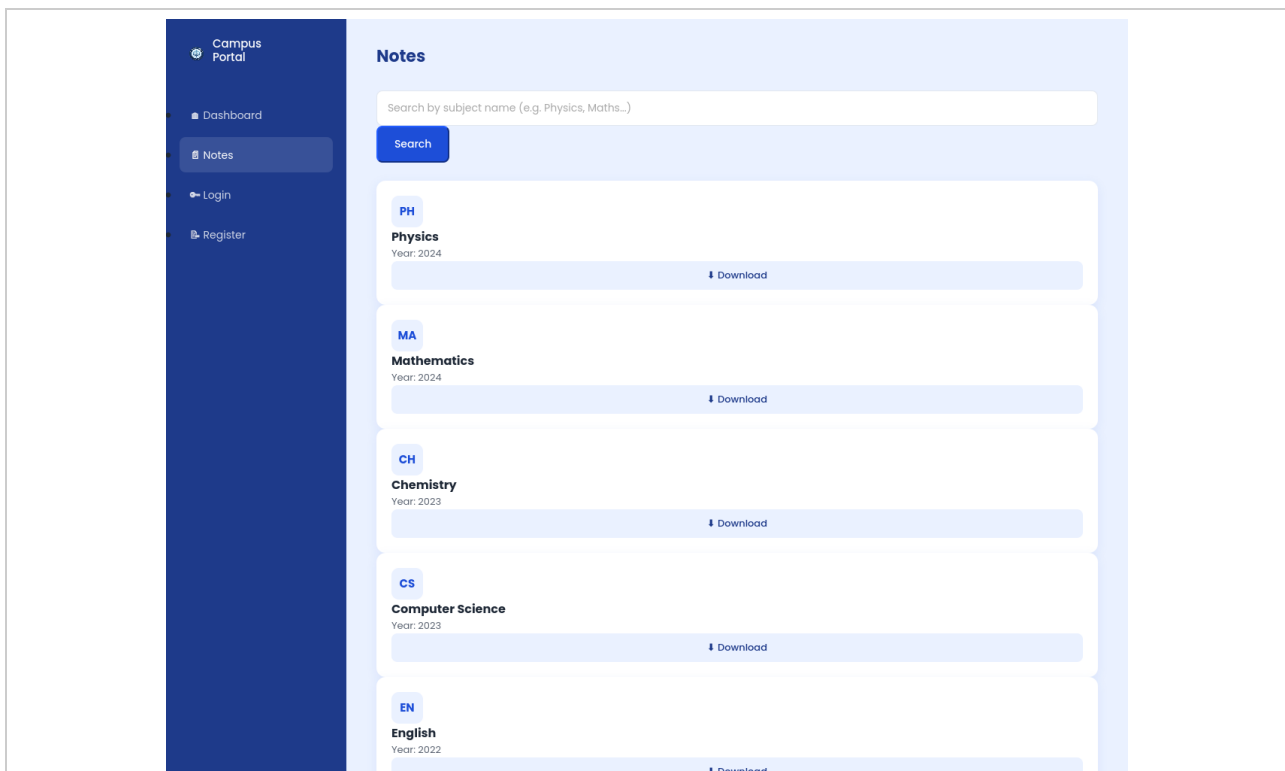


Figure 4 — Notes repository with live subject search and download cards

3.4 Note Upload with Dual-side Validation

upload.php accepts a subject name, year and a PDF file, and now also greets the logged-in student by name in the top bar. On the client side, script.js checks that the subject is non-empty, the year is realistic, and a file has actually been chosen — showing inline error messages and a shake animation on invalid fields before the form can submit. On the server side, PHP independently re-validates the same fields, rejects any file whose extension is not .pdf, renames the file with a timestamp prefix to avoid collisions, moves it into the uploads/ directory, and inserts a record into the notes table using a prepared statement. A Bootstrap modal confirms a successful upload.

Figure 5 — Upload Note form with subject, year and PDF file input

3.5 Navigation & Branding Clean-up

The authenticated sidebar was simplified by removing the Contact and Register links that previously appeared even for logged-in students, leaving only Dashboard, Notes and Upload Notes alongside Logout. The project's own crest-style logo (images/logo.png) now replaces the plain “LOGO” text placeholder used in the earlier prototype, appearing on the login card, the registration card and the sidebar brand mark. A related clean-up pass fixed a character-encoding (“mojibake”) issue that had corrupted the emoji icons used next to sidebar links and buttons, replacing them with numeric HTML entities that render correctly across browsers.

3.6 Security Measures

- Passwords are never stored in plain text — password_hash() with PASSWORD_BCRYPT is used at registration and password_verify() at login.
- session_regenerate_id() is called on every successful login and immediately after registration to defend against session fixation.
- Every SQL query in the application uses PDO prepared statements with bound parameters, preventing SQL injection.
- User-supplied text, including the student's username on the dashboard and notes pages, is passed through htmlspecialchars() before being echoed back into HTML, mitigating stored/reflected XSS.
- File uploads are restricted to the .pdf extension and renamed on the server before being stored.

4. Challenges and Solutions

Challenge	Solution
Keeping the JavaScript-only prototype (Phase 2 static HTML pages) working alongside the new PHP pages during the transition to a database-driven backend.	Client-side validation logic in script.js was kept unchanged and layered on top of the PHP forms, so both layers validate independently — PHP re-checks everything server-side regardless of what the browser already validated.

Challenge	Solution
Newly registered students had to log in a second time immediately after signing up, which felt redundant.	register.php now starts the session itself right after the INSERT succeeds (session_regenerate_id() + \$_SESSION['user_id']) and redirects straight to the dashboard, so registration doubles as an automatic first login.
The authenticated sidebar was still showing Contact and Register links to students who were already logged in, which was confusing and out of scope for Phase 3.	Those links were removed from the sidebar markup on every protected page (dashboard, notes, upload), leaving only the navigation a logged-in student actually needs.
Emoji icons used next to sidebar links and buttons displayed as garbled “mojibake” characters in some browser/encoding combinations.	The raw emoji characters were replaced with their numeric HTML entity equivalents (e.g. 🏠), which render consistently regardless of the file's saved encoding.
Preventing duplicate or unsafe uploaded file names from overwriting each other on the server.	Each uploaded file is renamed with a time()-based prefix before being moved into the uploads/ folder, guaranteeing a unique filename.
Guarding against SQL injection across every form (registration, login, upload).	All queries were rewritten to use PDO prepared statements with bound parameters instead of directly concatenating user input into SQL strings.
Making the notes search feel instant without reloading the page.	A pure JavaScript filter listens to the search input's keystrokes and toggles the CSS display of each note card in the DOM, rather than sending a new request to the server on every keystroke.
Ensuring a logged-out user cannot reach protected pages (dashboard, upload) by typing the URL directly.	A shared requireLogin() helper in includes/functions.php checks the session on every protected PHP page and redirects to the login page if no session exists.

5. Individual Contribution

Team Member	Contribution
D.I.A.A. Wijesekara (ITT/2024/121)	Database design (users, messages, notes tables), authentication module (register.php, login.php, logout.php) including the auto-login-after-registration flow and session security, and the project proposal document.
K.S. Liyanage (ITT/2024/126)	Frontend layout and Bootstrap styling, notes/past-papers repository page with the JavaScript search filter, upload form with client- and server-side validation, the Campus Portal branding/logo integration, and wireframes.

Both members jointly handled integration testing, dynamic-username personalisation across the dashboard, notes and upload pages, the sidebar navigation clean-up, README documentation, and preparation for the viva. A detailed verbal breakdown of individual understanding will be presented at the scheduled viva session, as required by the module guide.

6. Setup Instructions

Follow these steps to run the project locally using XAMPP:

1. Install XAMPP and start the Apache and MySQL modules from the XAMPP Control Panel.
2. Copy the project folder (Student_Resource_Hub_2) into the htdocs directory of your XAMPP installation.
3. Open phpMyAdmin at <http://localhost/phpmyadmin> in a browser.
4. Create the database by importing database.sql — this creates the student_resource_hub database along with the users, messages and notes tables.
5. Confirm the database credentials in includes/db.php match your local MySQL setup (default: host localhost, user root, empty password).
6. Visit http://localhost/Student_Resource_Hub_2/ in a browser to reach the login page.
7. Register a new student account — you will be signed in automatically and taken straight to the dashboard — or log in with an existing account to access the notes repository and upload page.

7. References

- PHP Manual — password_hash(): <https://www.php.net/manual/en/function.password-hash.php>
- PHP Manual — PDO Prepared Statements: <https://www.php.net/manual/en/pdo.prepared-statements.php>
- Bootstrap 5 Documentation: <https://getbootstrap.com/docs/5.3/>
- MDN Web Docs — JavaScript Guide: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
- Rajarata University of Sri Lanka, Department of ICT — ICT 1209 Mini Project Guide, 2024 Batch.
- Project GitHub Repository:
 1. https://github.com/Inuka-Asith30/Student_Resource_Hub
 2. https://github.com/Inuka-Asith30/Student_Resource_Hub_2.git